

ArcadePortal

Portal z grami przeglądarkowymi

Dokumentacja projektowa

Projektowanie i programowanie systemów internetowych I

Semestr letni 2024/2025

Przemysław Pyrzyński Michał Napora Maksymilian Wojtkowski

Spis treści

1	Opis funkcjonalny systemu	2
1.1	Moduł gier	2
1.2	System kont użytkowników	2
1.3	Ranking i osiągnięcia	2
1.4	Forum	2
1.5	Panel administratora	3
2	Opis technologiczny	3
2.1	Stos technologiczny	3
2.2	Architektura aplikacji	3
3	Schemat bazy danych (ERD)	4
3.1	Tabele własne	4
3.1.1	Users (rozszerzenie IdentityUser)	4
3.1.2	Games	4
3.1.3	GameSessions	4
3.1.4	Achievements	5
3.1.5	UserAchievements	5
3.1.6	Threads	5
3.1.7	Posts	5
3.2	Relacje między tabelami	6
4	Wdrożone zagadnienia kwalifikacyjne	6
5	Instrukcja uruchomienia	7
5.1	Uruchomienie lokalne	7
5.2	Uruchomienie przez Docker	7
6	Wnioski projektowe	8
6.1	Osiągnięte cele	8
6.2	Napotkane trudności	8
6.3	Możliwe rozszerzenia	8
	Opis zagadnień kwalifikacyjnych	9
	Opis szczegółowy każdego zagadnienia	11

1. Opis funkcjonalny systemu

ArcadePortal to portal z grami przeglądarkowymi umożliwiający użytkownikom rejestrację kont, rywalizację w grach zaimplementowanych w JavaScript oraz śledzenie wyników na globalnym rankingu.

1.1. Moduł gier

System obsługuje następujące gry przeglądarkowe:

- **Snake** — klasyczna gra węża na siatce 24×24 , rysowana na elemencie canvas
- **Tetris** — zaawansowany Tetris z grawitacją, przyspieszeniem i podglądem następnego klocka
- **Flappy Bird** — gra zręcznościowa z fizyką grawitacji i generowaniem rur
- **Pac-Man** — klasyczny Pac-Man z duchami, power pelletami i systemem poziomów
- **Blackjack** — karciana gra blackjack z pełną mechaniką asów i krupiera

1.2. System kont użytkowników

- Rejestracja z walidacją danych i emailem powitalnym
- Logowanie przez nazwę użytkownika lub adres email
- Profil użytkownika z historią rozgrywek i statystykami
- Zmiana nazwy użytkownika, avatara i hasła
- Reset hasła przez email z tokenem jednorazowym
- System ról: *Admin* i *User*

1.3. Ranking i osiągnięcia

- Globalny ranking oparty na sumie najlepszych wyników per gra
- Top 10 per gra z cachowaniem wyników (60 sekund)
- System odznak przyznawanych automatycznie po spełnieniu warunków
- Powiadomienia toast o nowych odznakach po zakończeniu gry

1.4. Forum

- Wątki ogólne i przypisane do konkretnej gry
- Tworzenie wątków, dodawanie odpowiedzi, edycja i usuwanie własnych postów
- Niezalogowani mogą czytać, zalogowani mogą pisać
- Moderacja przez administratorów

1.5. Panel administratora

- Zarządzanie użytkownikami: zmiana ról, blokowanie kont
- Zarządzanie grami: dodawanie, edycja, włączanie/wyłączanie
- Zarządzanie odznakami: tworzenie warunków per gra
- Usuwanie postów innych użytkowników
- Statystyki globalne: liczba użytkowników, sesji, wątków, postów

2. Opis technologiczny

2.1. Stos technologiczny

Warstwa	Technologia			Opis
Backend	ASP.NET	Core	8	Framework MVC — kontrolery, routing, autoryzacja
ORM	Entity Framework		Core 8	Mapowanie obiektowo-relacyjne, migracje, LINQ
Baza danych	SQLite			Lekka baza plikowa, idealna do projektu zaliczeniowego
Identity	ASP.NET	Core	Identity	System ról, hashowanie haseł, logout, tokeny
Cache	IMemoryCache			Cachowanie rankingów w pamięci serwera
Frontend	HTML5 + CSS3			Razor Views, responsywny layout (RWD)
CSS Framework	Bootstrap		5	Grid system, komponenty UI
JavaScript	Vanilla JS		(ES6+)	Logika gier, fetch API, animacje
Gry	HTML5 Canvas		API	Renderowanie gier: Snake, Tetris, Flappy, Pac-Man
Mailing	MailKit + SMTP			Wysyłka maili: rejestracja, reset hasła, odznaki
Lokalizacja	IStringLocalizer		/	Obsługa języków PL i EN
Konteneryzacja	Docker + Docker Compose			Pakowanie aplikacji z Mailpittem

2.2. Architektura aplikacji

Aplikacja zbudowana jest w architekturze MVC (Model-View-Controller) z dodatkową warstwą serwisów:

- **Models** — encje EF Core mapowane na tabele bazy danych
- **DTOs** — obiekty transferu danych między kontrolerami a widokami
- **Controllers** — obsługa requestów HTTP, walidacja, logika routingu

- **Services** — logika biznesowa (AchievementService, MailKitEmailService)
- **Views** — szablony Razor (.cshtml) z kodem HTML/CSS
- **Data** — ApplicationDbContext, DbSeeder, migracje EF Core

3. Schemat bazy danych (ERD)

Baza danych oparta na SQLite zawiera 7 tabel własnych oraz tabele zarządzane przez ASP.NET Core Identity (AspNetUsers, AspNetRoles, AspNetUserRoles i inne).

3.1. Tabele własne

3.1.1. Users (rozszerzenie IdentityUser)

Kolumna	Typ	Opis
Id	PK string	Klucz główny (GUID generowany przez Identity)
UserName	string	Unikalna nazwa użytkownika
Email	string	Adres email użytkownika
PasswordHash	string	Hash hasła (PBKDF2, zarządzany przez Identity)
AvatarUrl	string?	Opcjonalny URL avatara użytkownika
CreatedAt	DateTime	Data i czas rejestracji (UTC)

3.1.2. Games

Kolumna	Typ	Opis
Id	PK int	Klucz główny
Slug	string	Unikalny identyfikator tekstowy (np. "snake")
Title	string	Tytuł gry wyświetlany użytkownikom
Description	string?	Opcjonalny opis gry
ThumbnailUrl	string?	URL miniaturki gry
IsActive	bool	Czy gra jest widoczna dla graczy
CreatedAt	DateTime	Data dodania gry (UTC)

3.1.3. GameSessions

Kolumna	Typ	Opis
Id	PK int	Klucz główny
UserId	FK string	Referencja do Users
GameId	FK int	Referencja do Games

Kolumna	Typ	Opis
Score	int	Wynik zdobyty w tej sesji
DurationSeconds	int	Czas trwania rozgrywki w sekundach
PlayedAt	DateTime	Data i czas rozgrywki (UTC)

3.1.4. Achievements

Kolumna	Typ	Opis
Id	PK int	Klucz główny
GameId	FK int	Referencja do Games (odznaka należy do gry)
Name	string	Nazwa odznaki wyświetlana użytkownikowi
Description	string	Opis warunku zdobycia odznaki
IconUrl	string?	Emoji lub URL ikony odznaki
Condition	string	Warunek w formacie "score>=100" lub "duration>=60"

3.1.5. UserAchievements

Kolumna	Typ	Opis
Id	PK int	Klucz główny
UserId	FK string	Referencja do Users
AchievementId	FK int	Referencja do Achievements
UnlockedAt	DateTime	Data i czas odblokowania odznaki (UTC)

3.1.6. Threads

Kolumna	Typ	Opis
Id	PK int	Klucz główny
Title	string	Tytuł wątku (max 200 znaków)
UserId	FK string	Autor wątku — referencja do Users
GameId	FK int?	Opcjonalna referencja do Games (null = wątek ogólny)
IsPinned	bool	Czy wątek jest przypięty przez moderatora
CreatedAt	DateTime	Data i czas utworzenia wątku (UTC)

3.1.7. Posts

Kolumna	Typ	Opis
Id	PK int	Klucz główny
ThreadId	FK int	Referencja do Threads
UserId	FK string	Autor posta — referencja do Users
Body	string	Treść posta (max 5000 znaków)
CreatedAt	DateTime	Data i czas utworzenia posta (UTC)
UpdatedAt	DateTime?	Data ostatniej edycji (null = nie edytowano)

3.2. Relacje między tabelami

Tabela A	Relacja	Tabela B	Opis
Users	1 : N	GameSessions	Jeden użytkownik może mieć wiele sesji gry
Games	1 : N	GameSessions	Jedna gra może mieć wiele sesji
Users	1 : N	UserAchievements	Jeden użytkownik może mieć wiele odznak
Achievements	1 : N	UserAchievements	Jedna odznaka może być odblokowana przez wielu
Games	1 : N	Achievements	Jedna gra definiuje wiele odznak
Users	1 : N	Threads	Jeden użytkownik może tworzyć wiele wątków
Games	0..1 : N	Threads	Wątek opcjonalnie przypisany do gry
Threads	1 : N	Posts	Jeden wątek zawiera wiele postów
Users	1 : N	Posts	Jeden użytkownik może pisać wiele postów

4. Wdrożone zagadnienia kwalifikacyjne

#	Zagadnienie	Implementacja
1	Framework MVC	ASP.NET Core 8 MVC — kontrolery, modele, widoki Razor
2	Framework CSS	Bootstrap 5 — grid, komponenty; własny system CSS z variables
3	Baza danych	SQLite przez Entity Framework Core 8
4	Cache	IMemoryCache — cachowanie rankingów i leaderboardów
5	Dependency manager	NuGet — zarządzanie pakietami .NET

#	Zagadnienie	Implementacja
6	HTML	Razor Views (.cshtml) — szkielet aplikacji
7	CSS	Własne pliki CSS per moduł (auth, forum, game, admin...)
8	JavaScript	Logika gier w Vanilla JS, sterowanie, pętla gry
9	Routing	Atrybutowy i konwencjonalny routing MVC, pretty URLs
10	ORM	Entity Framework Core — DbContext, migracje, LINQ
11	Uwierzytelnianie	ASP.NET Core Identity — role, hasła, logout, tokeny
12	Lokalizacja	IStringLocalizer / IViewLocalizer, pliki .resx, PL/EN
13	Mailing	MailKit + SMTP — rejestracja, reset hasła, odznaki
14	Formularze	Razor Forms z walidacją, AntiForgeryToken
15	Asynchroniczne interakcje	Fetch API — zapis wyników, asynchroniczne zapytania
16	Konsumpcja API	Własne REST API /api/scores konsumowane przez JS gier
17	Publikacja API	Endpoint /api/scores (POST) z autoryzacją i CSRF
18	RWD	Bootstrap Grid + media queries, responsywny layout
19	Logger	ILogger<T> we wszystkich kontrolerach i serwisach
20	Deployment	Docker + Docker Compose z Mailpit i volume na bazę

5. Instrukcja uruchomienia

5.1. Uruchomienie lokalne

Wymagania: .NET 8 SDK, Mailpit

1. Sklonuj repozytorium: `git clone https://github.com/Przemek738/SystemyInternetowe\_Projekt`
2. Zainstaluj Mailpit i uruchom: `mailpit.exe`
3. W katalogu projektu: `dotnet ef database update`
4. Uruchom aplikację: `dotnet run`
5. Otwórz przeglądarkę: <http://localhost:5000>
6. Skrzynka mailowa: <http://localhost:8025>

5.2. Uruchomienie przez Docker

Wymagania: Docker Desktop

1. Sklonuj repozytorium i przejdź do katalogu projektu
2. Uruchom: `docker compose up -build`

3. Aplikacja dostępna pod: <http://localhost:8080>

4. Skrzynka mailowa: <http://localhost:8025>

Przy pierwszym uruchomieniu seeder automatycznie tworzy role, konto admina, gry z odznakami oraz 10 testowych użytkowników z losowymi wynikami.

6. Wnioski projektowe

6.1. Osiągnięte cele

- Zrealizowano portal z 5 grami przeglądarkowymi w pełni zintegrowanymi z systemem zapisywania wyników
- Zaimplementowano kompletny system kont z rolami, uwierzytelnianiem i resetem hasła przez email
- Stworzono forum dyskusyjne z moderacją i wątkami przypisanymi do gier
- Skonteneryzowano aplikację przez Docker z pełną izolacją środowiska

6.2. Napotkane trudności

- Integracja własnego systemu menu gier z jednym widokiem `Play.cshtml` wymagała przemyślenia architektury JS
- Konfiguracja ASP.NET Core Identity z własnym modelem User wymagała znajomości cyklu życia tokenów i claims

6.3. Możliwe rozszerzenia

- Dodanie OAuth2 (logowanie przez Google/GitHub)
- System turniejów i rozgrywek wieloosobowych
- Migracja bazy danych z SQLite na PostgreSQL dla środowiska produkcyjnego

Opis zagadnień kwalifikacyjnych

W tej sekcji opisujemy szczegółowo każde z 20 zagadnień kwalifikacyjnych — gdzie jest zaimplementowane w projekcie oraz jak można sprawdzić że działa.

#	Zagadnienie	Gdzie w projekcie	Jak sprawdzić
1	Framework MVC	Controllers/*.cs, Program.cs, Views/*.cshtml	Wejdź na /Home, /Game, /Forum — każda strona to inny kontroler
2	Framework CSS	Views/Shared/_Layout.cshtml (Bootstrap CDN), wwwroot/css/*.css	Otwórz DevTools → Elements → sprawdź klasy Bootstrap i własne
3	Baza danych	Data/AppDbContext.cs, appsettings.json, arcade.db	Otwórz plik arcade.db w DB Viewerze w Visual Studio
4	Cache	Controllers/GameController.cs (GetOrCreateAsync), LeaderboardController.cs	Zagraj grę, sprawdź wynik — przez 60s top 10 się nie zmienia
5	Dependency manager	ArcadeProject.csproj (<PackageReference>), Dockerfile	Otwórz .csproj — widać wszystkie paczki NuGet
6	HTML	Views/**/*.cshtml	— Ctrl+U na dowolnej stronie — widać wygenerowany HTML
7	CSS	wwwroot/css/gamePlay.css, auth.css, forum.css, admin.css, leaderboard.css, profile.css	DevTools → Network → CSS — każda strona ładuje swój plik CSS
8	JavaScript	wwwroot/js/gamePlay.js, snake.js, tetris.js, flappy.js, pacman.js, blackjack.js	DevTools → Sources → wwwroot/js/
9	Routing	Program.cs (MapControllerRoute), Controllers/GameController.cs ([HttpPost("/api/scores")])	Przejdź na /Game/Play/snake — "snake" to parametr z URL
10	ORM	Data/AppDbContext.cs, Models/*.cs, Migrations/	Folder Migrations/ — każda migracja to zmiana w bazie
11	Uwierzytelnianie	Controllers/AccountController.cs Program.cs (AddIdentity), Data/DbSeeder.cs	Wejdź na /Admin bez logowania — przekieruje na /Account/Login
12	Lokalizacja	Controllers/CultureController.cs Resources/**/*.resx	Kliknij EN w navbarze — strona główna zmienia się na angielski

#	Zagadnienie	Gdzie w projekcie	Jak sprawdzić
13	Mailing	Services/MailKitEmailService.cs	Zarejestruj konto → sprawdź http://localhost:8025
14	Formularze	Views/Account/Register.cshtml Views/Forum/Create.cshtml DTOs/AccountDtos.cs	Spróbuj zarejestrować się z pustymi polami — pojawią się błędy
15	Async interakcje	wwwroot/js/gamePlay.js (funkcja saveScore() z fetch())	DevTools → Network → za-graj grę → widać POST /api/scores
16	Konsumpcja API	wwwroot/js/gamePlay.js (fetch POST /api/scores)	Jak wyżej — JS konsumuje endpoint wystawiony przez kontroler
17	Publikacja API	Controllers/GameController.cs (metoda SaveScore, [HttpPost("/api/scores")])	DevTools → kliknij request /api/scores → Preview → JSON
18	RWD	wwwroot/css/gamePlay.css (@media max-width: 900px), _Layout.cshtml (navbar-expand-md)	DevTools → Ctrl+Shift+I → symuluj iPhone → sprawdź układ
19	Logger	Controllers/HomeController.cs, GameController.cs, AccountController.cs, Services/MailKitEmailService.cs	docker compose logs app -f → zaloguj się → widać logi
20	Deployment	Dockerfile, docker-compose.yml, .dockerignore	Na innym komputerze: docker compose up -build → działa bez .NET

Opis szczegółowy każdego zagadnienia

1. Framework MVC Projekt używa ASP.NET Core 8 MVC. Każda strona to kontroler, który obsługuje żądania i zwraca widok. Na przykład `HomeController` obsługuje stronę główną, `GameController` obsługuje listę gier i uruchamianie gry, `ForumController` obsługuje forum itd. Routing jest skonfigurowany w `Program.cs`.

2. Framework CSS Używamy Bootstrap 5, który jest dołączony przez CDN w pliku `_Layout.cshtml`. Bootstrap obsługuje responsywny grid i nawigację. Oprócz Bootstrap mamy własne pliki CSS dla każdego modułu — osobny plik dla strony gry, forum, logowania, panelu admina itd. Dzięki temu style się nie mieszają i każda strona ładuje tylko to, czego potrzebuje.

3. Baza danych Używamy SQLite, bo jest prosty w użyciu i nie wymaga osobnego serwera. Baza to jeden plik `arcade.db`. W Dockerze ten plik jest trzymany na osobnym volume, żeby nie zginął przy rebuild.

4. Cache `IMemoryCache` jest wstrzyknięty do `GameController` i `LeaderboardController`. Ranking top 10 per gra jest cachowany przez 60 sekund. Globalny ranking jest cachowany przez 2 minuty. Po zapisaniu nowego wyniku cache jest usuwany przez `_cache.Remove()`, żeby przy następnym wejściu dane były świeże.

5. Dependency manager NuGet jest standardowym menedżerem pakietów dla .NET. Wszystkie zależności (EF Core, Identity, MailKit, Bootstrap itp.) są zapisane w pliku `ArcadeProject.csproj` w sekcji `PackageReference`. Komenda `dotnet restore` pobiera je automatycznie. W `Dockerfile` `restore` jest jako osobna warstwa, co przyspiesza buildy.

6. HTML Używamy Razor Views — pliki `.cshtml`, które są szablonami HTML z możliwością wstawiania kodu C# przez składnię `@`. Każdy kontroler ma swój folder w `Views/`. Plik `_Layout.cshtml` jest wspólnym szkieletem dla wszystkich stron — zawiera navbar, footer i miejsca na sekcje `Scripts` i `Styles`.

7. CSS Oprócz Bootstrap mamy własne pliki CSS. Globalny styl jest w `site.css`. Każda sekcja ma swój plik — `auth.css` dla logowania i rejestracji, `forum.css` itd. Używamy CSS Custom Properties (zmienne) dla kolorów motywu.

8. JavaScript Logika gier jest napisana w czystym JavaScript (ES6+) bez frameworków. Plik `gamePlay.js` to silnik strony gry — obsługuje menu, zapis wyników przez fetch i powiadomienia o odznakach. Każda gra ma swój plik (`snake.js`, `tetris.js` itd.), który implementuje funkcję `window.gameStart()` wywoływaną przez `gamePlay.js` po kliknięciu START. Komunikacja z serwerem odbywa się przez Fetch API.

9. Routing W `Program.cs` mamy standardowy routing MVC. Dzięki temu URL `/Game/Play/snake` przekazuje "snake" jako parametr slug do metody `Play()` w `GameController`.

10. ORM Entity Framework Core 8 jest naszym ORM. `AppDbContext` dziedziczy po `IdentityDbContext` i definiuje `DbSet`y dla wszystkich encji. Relacje (np. kaskadowe usuwanie) są skonfigurowane w metodzie `OnModelCreating`. Zapytania piszemy w LINQ. Migracje są w folderze `Migrations/`.

11. Uwierzytelnianie ASP.NET Core Identity zarządza użytkownikami i rolami. Mamy dwie role: *Admin* i *User* tworzone przez seeder przy starcie. Hasła są hashowane algorytmem PBKDF2. Po 5 nieudanych próbach konto jest blokowane na 5 minut. Atrybut `[Authorize(Roles="Admin")]` na `AdminController` sprawia, że tylko admini mają dostęp do panelu. Logowanie działa przez `username` lub `email`.

12. Lokalizacja Obsługujemy dwa języki: polski i angielski. W `navbarze` jest przycisk `PL/EN`, który wysyła `POST` do `CultureController` i zapisuje wybrany język w ciasteczku na rok. Tłumaczenia są w plikach `.resx` w folderze `Resources/` — osobny plik per kontroler i widok. W widokach używamy `@inject IViewLocalizer Loc` i `@Loc["klucz"]` zamiast hardcodowanych tekstów.

13. Mailing Do wysyłania emaili używamy biblioteki `MailKit`. Stworzyliśmy interfejs `IService` z metodami `SendWelcomeAsync`, `SendPasswordResetAsync` i `SendAchievementAsync`. Implementacja `MailKitEmailService` łączy się przez `SMTP` do `Mailpit` (lokalnie i w `Dockerze`) lub zewnętrznego serwera. Jeśli wysyłka się nie uda, błąd jest logowany, ale nie przerywa działania aplikacji (`try/catch`).

14. Formularze Formularze w aplikacji używają `Razor Tag Helpers` — `asp-for`, `asp-action`, `asp-validation-for`. Walidacja jest definiowana przez atrybuty w klasach `DTO`, np. `[Required]`, `[MinLength(6)]`, `[EmailAddress]`. Każdy formularz `POST` jest zabezpieczony przez `[ValidateAntiForgeryToken]`, który chroni przed atakami `CSRF`. Błędy walidacji wracają do widoku przez `ModelState`.

15. Asynchroniczne interakcje Po zakończeniu gry `JavaScript` wywołuje `fetch()` `POST /api/scores` bez przeładowania strony. Wysyła `JSON` z wynikiem i czasem gry. Po odebraniu odpowiedzi aktualizuje pasek statusu (wynik, rekord osobisty) i pokazuje powiadomienia o odznakach jako animowane elementy `HTML` dodawane do `body`. Wszystko dzieje się bez `reload`.

16. Konsumpcja API Plik `gamePlay.js` konsumuje endpoint `/api/scores` przez natywne `fetch()` `API` przeglądarki. Wysyła `JSON` `body` z `gameSlug`, `score` i `durationSeconds`. Dołącza też token `CSRF` z ukrytego pola formularza w nagłówku `RequestVerificationToken`. Obsługuje błędy — jeśli serwer zwróci `401` (niezalogowany), wyświetla komunikat bez `crashowania` gry.

17. Publikacja API Metoda `SaveScore()` w `GameController` jest wystawiona jako `REST` endpoint. Ma atrybut `[HttpPost("/api/scores")]` i `[ValidateAntiForgeryToken]`. Sprawdza czy użytkownik jest zalogowany (`401` jeśli nie) i czy gra istnieje (`404` jeśli nie). Zapisuje sesję do bazy, wywołuje `AchievementService` i zwraca `JSON`: `{ message, score, newAchievements: [...] }`.

18. RWD Strona jest responsywna dzięki Bootstrap Grid i własnym media queries. Strona gry używa CSS Grid, który automatycznie przełącza się z 2 kolumn (gra + sidebar) na 1 kolumnę poniżej 900px. Nawigacja korzysta z `navbar-expand-md` Bootstrap — menu zwija się do hamburgera na telefonach. Siatka kart gier używa `auto-fill` z `minmax(260px, 1fr)`, co automatycznie dobiera liczbę kolumn.

19. Logger Każdy kontroler i serwis ma wstrzyknięty `ILogger<T>` przez konstruktor. Logujemy informacje przy kluczowych akcjach: załadowanie strony głównej, logowanie, rejestracja, wejście w grę, zapis wyniku, przyznanie odznaki. Błędy wysyłki emaila logujemy przez `LogError`. W Dockerze logi widać przez komendę `docker compose logs app -f`.

20. Deployment Aplikacja jest skonteneryzowana w Dockerze. `Dockerfile` używa multi-stage build — najpierw obraz z SDK do kompilacji, potem lżejszy obraz `aspnet` do uruchomienia. `docker-compose.yml` uruchamia dwa kontenery: aplikację na porcie 8080 i Mailpit na portach 8025/1025. Baza SQLite jest na volume `db-data`, który przeżywa rebuildy. Na innym komputerze wystarczy Docker Desktop i komenda `docker compose up -build`.